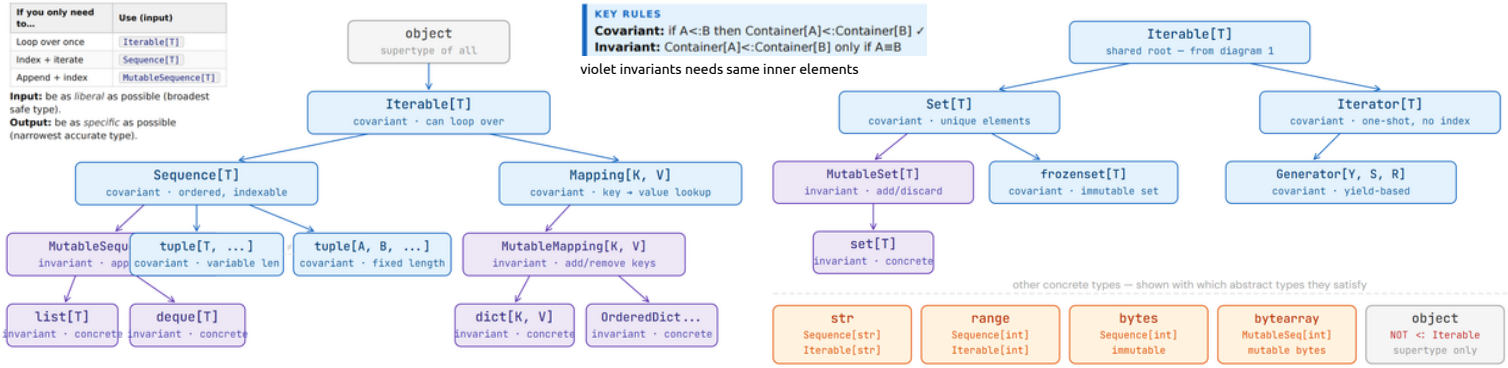
SUBTYPE HIERARCHIES

If you only need to...	Use (input)
Loop over once	`Iterable[T]`
Index + iterate	`Sequence[T]`
Append + index	`MutableSequence[T]`

Input: be as *liberal* as possible (broadest safe type).
Output: be as *specific* as possible (narrowest accurate type).



KEY RULES
Covariant: if A<:B then Container[A]<:Container[B] ✓
Invariant: Container[A]<:Container[B] only if A≡B

violet invariants needs same inner elements

TYPE	IS A SUBTYPE OF	KEY EXAM RULE
`set[T]`	`MutableSet[T]`, `Set[T]`, `Iterable[T]`, `object`	`set[A] <: set[B]` only if A = B (invariant)
`frozenset[T]`	`Set[T]`, `Iterable[T]`, `object`	`frozenset[A] <: frozenset[B]` if A <: B (covariant)
`dict[K, V]`	`MutableMapping[K,V]`, `Mapping[K,V]`, `Iterable[K]`, `object`	`dict` is Iterable over its *keys*, not values
`list[T]`	`MutableSequence[T]`, `Sequence[T]`, `Iterable[T]`, `object`	`list[A] <: list[B]` only if A = B (invariant)
`tuple[T, ...]`	`Sequence[T]`, `Iterable[T]`, `object`	`tuple[A,...] <: tuple[B,...]` if A <: B (covariant)
`tuple[A, B]`	`Sequence[A|B]`, `Iterable[A|B]`, `object`	`tuple[A,B] <: tuple[C,D]` if A<:C and B<:D (covariant per position)
`str`	`Sequence[str]`, `Iterable[str]`, `object`	`str` is an immutable sequence of `str` characters
`object`	`object` (only itself)	`object` is the top of the hierarchy — it flows UP, nothing passes it DOWN
`Sequence[T]`	`Iterable[T]`, `object`	Sequence is broader than list — you cannot pass Sequence where list is expected
`Iterable[T]`	`object`	Iterable is the broadest collection type — nothing collection-ish is a supertype

QUICK SUBTYPE RELATIONSHIPS

```
bool            <: int
list[T]         <: MutableSequence[T] <:
Sequence[T] <: Iterable[T]
tuple[A,B]      <: Sequence[A|B]    (fixed, covariant)
tuple[T,...]    <: Sequence[T]      (variable, covariant)
A               <: A | B            (for any A, B)
A               <: object           (for any A)
Sequence[A]     <: Sequence[B]      if A <: B (covariant)
list[A]
<: list[B]          ONLY if A ≡ B  (invariant)

NOT subtypes:
list[A]         ∉  tuple[A,...]
tuple[A,A]      ∉  tuple[A,...]
object          ∉  anything specific
Iterable[T]     ∉  Sequence[T]   (it's broader)
```

CALLABLE ANNOTATIONS & FUNCTION SUBTYPING

`Callable[[ArgType], ReturnType]`

```
# function taking Pikachu, returning Pikachu
f: Callable[[Pikachu], Pikachu]
```

PICO RULE — FOR F_CHILD <: F_PARENT
Parameters **I**nvert (contravariant):
child accepts *broader or same* input than parent
i.e., parent_param <: child_param

COvariant returns:
child returns *narrower or same* output
i.e., child_return <: parent_return

MVC ARCHITECTURE

Layer	Contains	Never contains
Model	State, rules, validation, domain logic	input(), print(), any I/O
View	All input() and print() calls, formatting	Game rules, state mutations
Controller	Main loop, calls Model + View, orchestrates	Raw I/O, direct business logic

SRP — SINGLE RESPONSIBILITY PRINCIPLE

A class must have **exactly one reason to change**.

Ask: could two different external pressures independently force this class to change? If yes → SRP violation. Name each responsibility and who would cause it to change.

ISP — INTERFACE SEGREGATION PRINCIPLE

Clients must not be forced to depend on methods they don't use. Split fat interfaces into smaller focused ones.

LSP — ALL 7 BEHAVIORAL RULES

Rule 1 **Parameters must be contravariant**
Child accepts *at least as broad* input as parent. Cannot require a narrower type.

```
Parent: process(x: Animal)      # accepts any Animal
Child:  process(x: Dog)  ← VIOLATION # rejects Cat
```

Rule 2 **Return types must be covariant**
Child returns *at least as specific* a type. Cannot return something broader.

```
Parent: get_pet() -> Dog            # promises a Dog
Child:  get_pet() -> Animal  ← VIOLATION # could return Cat
```

Rule 3 **Exceptions must be subtypes of parent's exceptions**
Child may raise `ValueError` subclass if parent raises `ValueError`. Must not throw unrelated exceptions callers don't catch.

```
class CodeLengthError(ValueError): pass  ← OK
# CodeLengthError is a subtype of ValueError
```

Rule 4 **Preconditions must not be strengthened**
Precondition = what must be true before calling. Child cannot demand *more* from the caller.

```
Parent: sqrt(x) where x >= 0     # precondition
Child:  sqrt(x) where x >= 1  ← VIOLATION
# caller passing 0.5 satisfies parent, fails child
```

Rule 5 **Postconditions must not be weakened**
Postcondition = what is guaranteed after call. Child cannot deliver *less* than parent promised.

```
Parent: draw() -> Card           # promises non-None
Child:  draw() -> Card | None  ← VIOLATION
# callers doing .rank will crash on None
```

Rule 6 **Invariants must not be violated**
Invariant = property always true throughout object lifetime. Child must not introduce state that breaks it.

```
class ImmutablePoint:
    """Invariant: x and y never change"""
class MutablePoint(ImmutablePoint):
    def set_x(self, v): self._x = v  ← VIOLATION
```

Rule 7 **History rule must not be violated**
Parent-visible state transitions in child must be achievable using only parent's methods. Child cannot add state escape hatches.

```
class Counter:   # count only ever increases
    def increment(self): self._count += 1
class ResettableCounter(Counter):
    def reset(self): self._count = 0  ← VIOLATION
# "go to 0" is unreachable via Counter.increment
```

RULE 6 VS RULE 7
Rule 6: child breaks a state the parent explicitly declared impossible (adds setter to "immutable").
Rule 7: child adds a new state transition that no combination of parent methods could ever produce.

DIP — DEPENDENCY INVERSION PRINCIPLE

High-level modules must not depend on low-level concretions. Both depend on an abstraction.

```
# VIOLATION: Bank hardcodes concrete vault
class Bank:
    def __init__(self):
        self.vault = CombinationLockVault("00000") ←

# FIX: inject abstraction
class Vault(Protocol): ...   # abstraction
class Bank:
    def __init__(self, vault: Vault):  # injected
        self.vault = vault
```

```
class Animal:
    def __init__(self, name: str):
        self.name = name
        self.is_alive = True
```

```
class Dog(Animal):
    def __init__(self, name: str, breed: str):
        super().__init__(name)  # sets name, is_alive
        self.breed = breed      # then add own attrs
```

INHERITANCE, MRO & SUPER()

MRO: Python searches Child → Parent → Grandparent → ... → object.

`self.method()` → always dispatches from the **actual runtime class** of the object, even if called inside a parent method.

`super().method()` → starts from the **next class in MRO** relative to where super() is written.

```
class A:
    def label(self): return "A"
    def describe(self): return f"I am {self.label()}"

class B(A):
    def label(self): return "B"   # overrides A

B().describe()
# B has no describe → uses A.describe()
# BUT self.label() → B.label() → "I am B"  ← NOT "I am A"!
```

ALWAYS CALL SUPER().__INIT__() FIRST
Missing it means parent attributes are never set → AttributeError. In a 3-level chain A→B→C, if B skips super(), A's `__init__` is never called — even if C calls super() correctly.

LSP — LISKOV SUBSTITUTION PRINCIPLE

If S<:T, objects of type S must be substitutable for T without breaking correct behavior.

① *"Do not require more than originally needed"*
② *"Do not give less than originally promised"*
③ *"Stay true to your roots"*

OCP — OPEN/CLOSED PRINCIPLE

Adding new behavior = **add new code**, never modify existing code.

```
# VIOLATION: must edit for every new type
if unit_type == "Soldier": ...
elif unit_type == "Archer": ...  ← OCP violation

# FIX: polymorphism via Protocol
class AttackValidator(Protocol):
    def can_attack(self, src, tgt) -> bool: ...
class SoldierValidator:
    def can_attack(self, src, tgt): ...
class GunnerValidator:   # new — zero edits elsewhere
    def can_attack(self, src, tgt): ...
```

ISINSTANCE = OCP VIOLATION
Using `isinstance(obj, SubClass)` to branch on type is the same anti-pattern. Let the object handle its own behavior via a method. Every new subclass forces editing the isinstance chain.

DESIGN PATTERN QUICK COMPARISON

Pattern	Core idea	Uses	OCP role
Strategy	Swap algorithms at runtime	Composition + Protocol	Add new strategy = new class
Template Method	Fixed skeleton, custom steps	Inheritance + ABC	Add variant = new subclass
State	Behavior changes with state	Composition + Protocol	Add state = new class
Observer	Notify unknown observers	Composition (list of observers)	Add observer = new class
Adapter	Convert incompatible interface	Composition (wraps adaptee)	Add adapter = new class

PYRIGHT STRICT MODE RULES

Rule	Bad	Good
All params annotated	`def f(x, y):`	`def f(x: int, y: int):`
Empty collection typed	`nums = []`	`nums: list[int] = []`
Return type inferred OK	—	Pyright infers from body

Static checking happens *before* execution (Pyright). Dynamic checking happens *during* execution (Python default). Type hints alone do nothing — you need to run Pyright.

OBJECT INVARIANTS & THE ALIASING TRAP

An invariant is a condition the object always guarantees. All methods (incl. inherited) must preserve it. RETURN COPY FROM OUTSIDE TO INSIDE DATA

```
# UNSAFE: new dict but same list objects inside!
return {k: self._mapping[k] for k in self._mapping}

# caller can then do:
m["A"].append(Student("X", "B"))  # breaks invariant!

# SAFE: copy the inner lists too
return {k: list(v) for k, v in self._mapping.items()}
```

🚩 RED FLAGS

Signal	Likely Violation
`if type=="a": elif type=="b":`	OCP
`isinstance(obj, SubClass)` chains	OCP
`self.x = ConcreteClass()` in `__init__`	DIP
Class has unrelated methods	SRP
Fat interface gives client irrelevant methods	ISP
Child method has narrower param type	LSP Rule 1
Child returns `X | None` where parent returns `X`	LSP Rule 5
Child raises unrelated exception	LSP Rule 3
Child tightens precondition (e.g. x≥1 when parent x≥0)	LSP Rule 4
Child adds setter to "immutable" parent	LSP Rule 6
Child adds reset/clear that parent can't do	LSP Rule 7
Returning internal list reference, not copy	Invariant break
Missing `super().__init__()`	AttributeError (runtime)
`print()` or `input()` in Model	MVC / SRP
`list[T]` param when only iterating	Annotation too narrow
`Iterable[T]` return when body returns tuple	Annotation too broad
`nums = []` without type	Pyright strict error

BINARY SEARCH (BYENARY_SEARCH) — BUG & FIX

Precondition: f must be a *pure, strictly increasing* function. f(l) ≤ f(r).

```
# BUGGY version (as in exam)
while l < r:
    m = (l + r) // 2
    if f(m) < y:   l = m      ← BUG: should be m+1
    elif f(m) > y: r = m      ← BUG: should be m-1
    else: return m
return None

# WHY IT BUGS: if f(m) < y and m == l,
# setting l=m leaves l unchanged → infinite loop
# Example: l=0,r=2,y=8,f=cube
#   m=1, f(1)=1<8 → l=1
#   m=1, f(1)=1<8 → l=1  (stuck forever!)

# FIXED version
def byenary_search(l: int, r: int, y: int,
                   f: Callable[[int], int]) -> int
| None:
    while l <= r:              # include endpoints
        m = (l + r) // 2
        if f(m) < y:   l = m + 1  ← excludes m
        elif f(m) > y: r = m - 1  ← excludes m
        else:          return m
    return None

# FAILING case (part a): l=0, r=2, y=8, f=cube
# PASSING case (part b): l=0, r=3, y=8, f=cube →
returns 2
```

MERGE SORT — ALGORITHM & RECURSION PATTERN

Divide and conquer: split list in half, sort each half recursively, merge sorted halves. O(n log n).

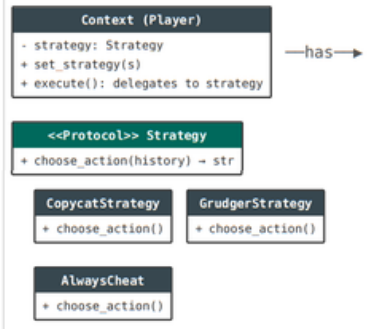
```
def merge_sort(lst: list[int]) -> list[int]:
    if len(lst) <= 1:
        return lst                  # base case
    mid = len(lst) // 2
    left  = merge_sort(lst[:mid])
    right = merge_sort(lst[mid:])
    return merge(left, right)

def merge(a: list[int], b: list[int]) -> list[int]:
    result: list[int] = []
    i = j = 0
    while i < len(a) and j < len(b):
        if a[i] <= b[j]:
            result.append(a[i]); i += 1
        else:
            result.append(b[j]); j += 1
    result.extend(a[i:])       # remaining
    result.extend(b[j:])
    return result

# Trace: merge_sort([4, 2, 7, 1])
#   → merge_sort([4,2]) + merge_sort([7,1])
#   → merge([2,4], [1,7]) → [1,2,4,7]
```

LSP relevance: A subclass sorting implementation must not weaken the postcondition "returns a sorted list of the same elements" — classic Rule 5 scenario.

```
from dataclasses import dataclass
from abc import ABC, abstractmethod
from collections.abc import Callable
from typing import Protocol
from typing import Literal
```

STRATEGY PATTERN — STRUCTURE & UML

When: behavior varies across objects of the same type, and you want to swap algorithms at runtime.

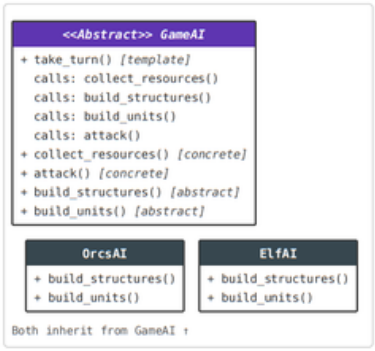



```
class Strategy(Protocol):
    def choose_action(self, history:
list[str]) -> str: ...

class Player:
    def __init__(self, strategy: Strategy):
        self._strategy = strategy
    def play(self, history: list[str]) -> str:
        return
self._strategy.choose_action(history)
```

vs Template Method: Strategy uses *composition*. Template Method uses *inheritance*. Strategy = swappable at runtime. Template = fixed at class definition.

TEMPLATE METHOD PATTERN — STRUCTURE & UML

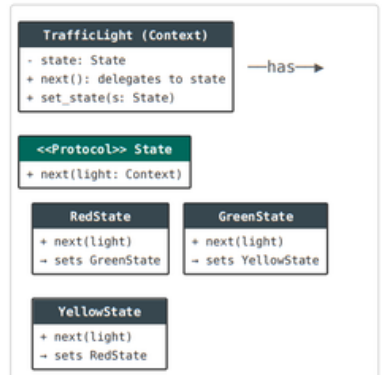
When: an algorithm has a fixed skeleton but some steps should be customizable by subclasses.




```
class GameAI(ABC):
    def take_turn(self):           # template
method
        self.collect_resources() # concrete
step
        self.build_structures()  # abstract
(hook)
        self.build_units()       # abstract
(hook)
        self.attack()            # concrete
step
    @abstractmethod
    def build_structures(self): ...
```

STATE PATTERN — STRUCTURE & UML

When: an object's behavior changes dramatically based on internal state (finite state machine).




```
class RedState:
    def next(self, light):
        light.set_state(GreenState())
# Each state owns its own transition logic
# Adding a new state = new class, no edits to
context
```

OBSERVER PATTERN — STRUCTURE & UML

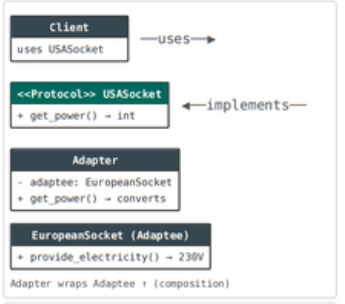
When: many objects need to react when one object changes state, without the subject knowing who they are.


```
class Observable:
    def __init__(self):
        self._observers: list = []
    def attach(self, o):
self._observers.append(o)
    def detach(self, o):
self._observers.remove(o)
    def notify(self, *args):
        for o in self._observers:
o.update(*args)

class Store(Observable):
    def restock_product(self):
        self.product_available = True
        self.notify("Product is now available!")
# Store never imports Customer. Loose coupling!
```

OCP connection: Adding a new observer type (e.g., EmailNotifier) requires zero changes to Store. Just create the class and attach it.

ADAPTER PATTERN — STRUCTURE & UML

When: you need to use an existing class but its interface doesn't match what clients expect.




```
class EuropeanToUSAAdapter:  # implements
USASocket
    def __init__(self, socket: EuropeanSocket):
        self._socket = socket
    def get_power(self) -> int:
        v = self._socket.provide_electricity()
        return int(v * (120 / 230))
```

Three parts: (1) Incompatible interface (already exists, inside adapter). (2) Compatible interface (expected by client, implemented by adapter). (3) Adapter class (converts between them).

ISINSTANCE() = OCP VIOLATION — EXPLAINED (Q9)

Every time you add a new subclass, any function using isinstance-branching must be edited. That violates OCP (closed for modification).

```
# VIOLATES OCP: adding FireTypeMove requires
editing this
def process_move(move: Move) -> str:
    if isinstance(move, ElectricTypeMove):
        return "ZAP"
    elif isinstance(move, PsychicTypeMove):
        return "PSYCH"
    # FireTypeMove → must edit THIS function

# OCP-COMPLIANT: each class handles itself
class Move(Protocol):
    def get_effect(self) -> str: ...

class ElectricTypeMove:
    def get_effect(self) -> str: return "ZAP"

class FireTypeMove:          # new — zero edits
needed
    def get_effect(self) -> str: return "BURN"

def process_move(move: Move) -> str:
    return move.get_effect()   # polymorphic
dispatch
```

```
└── Exception
    ├── ArithmeticError
    │   ├── FloatingPointError
    │   ├── OverflowError
    │   └── ZeroDivisionError
    ├── AssertionError
    ├── AttributeError
    ├── BufferError
    ├── EOFError
    ├── ExceptionGroup [BaseExceptionGroup]
    ├── ImportError
    │   └── ModuleNotFoundError
    ├── LookupError
    │   ├── IndexError
    │   └── KeyError
    ├── MemoryError
    ├── NameError
    │   └── UnboundLocalError
    ├── OSError
```

MRO OUTPUT TRACING — FULL TEMPLATE

Rule 1: `self.method()` always dispatches from the *actual runtime class* (the class used in the constructor), even when called from inside a parent's method body.

Rule 2: `super().method()` starts from the *next class in MRO after the class where super() is written* — not from the caller's runtime class.

MRO for single inheritance: Child → Parent → Grandparent → ... → object

```
# Given: DCSStudent(UPStudent(Student))
# MRO: DCSStudent → UPStudent → Student →
object

DCSStudent('Chippy', '1094-00012').identifier()

# STEP 1: DCSStudent has identifier() → run it
#   identifier = super().identifier()  → goes
to UPStudent
#   STEP 2: UPStudent has identifier() → run it
#     identifier = super().identifier()  → goes
to Student
#     STEP 3: Student.identifier():
#       return f"{self.label()}: {self.name}"
#       self.label() → runtime class =
DCSStudent!
#       DCSStudent.label() → "DCS Student"
#       → "DCS Student: Chippy"
#     back in UPStudent: f"[{student_no}]
{identifier}"
#     → "[1094-00012] DCS Student: Chippy"
#   back in DCSStudent: f"[DCS] {identifier}"
#   → "[DCS] [1094-00012] DCS Student: Chippy"
```

THE TRAP EVERY STUDENT FALLS INTO

Student.identifier() calls `self.label()`. Even though the method body is inside Student, `self` is still a DCSStudent object. So Python dispatches to DCSStudent.label() — not Student.label(). This is the core of every output-tracing question.

Step-by-step method for any trace:

1. Write out the MRO for the actual runtime class
2. For every method call, find the first class in MRO that defines it
3. When you hit `super()`, jump to the *next class in MRO from where super() is written*
4. Whenever you see `self.foo()`, restart MRO lookup from the runtime class
5. Track the return value upward through each stack frame

WALRUS OPERATOR := (ASSIGNMENT EXPRESSION)

`VAR := EXPR` evaluates EXPR, assigns to VAR, and also *returns* the value. Used in conditions so you can test and bind in one step.

```
# Without walrus:
word = input()
if word:
    process(word)

# With walrus (as in Shiritori code):
if current_word := self.prompt_word():
    # current_word is assigned AND tested in
one step
    self.play_word(current_word)
```

EXAM PATTERN RECOGNITION GUIDE

If the question shows...	Think...	Answer involves...
Multiple declarations × multiple usages table	Subtype hierarchy + variance	Trace R <: P for each cell
Class hierarchy + Callable[[X],X] passed in	PICO + double-apply rule	Check param contravariance + return <: param
"Does it violate SRP?"	Count reasons to change	Name each responsibility and who owns it
"Re-implement as MVC"	Separate I/O from logic	3 classes: Model (no I/O), View (all I/O), Controller (loop)
"Which SOLID principle violated?"	Red flag table	Name principle + why + fix with code
Child class docstring says different pre/ postcondition	LSP behavioral rules 4 or 5	Quote both docstrings, state which rule, explain client impact
"Explain how invariant can be violated"	Aliasing + mutable state escape	Show code path that mutates internal state from outside
Abstract class hierarchy + "add new gen"	OCP via inheritance or strategy	Add subclass only, no edits to existing classes
class X(Y): with new public method	LSP History Rule 7	The new method creates transitions unreachable via Y's interface
Output of class hierarchy code	MRO tracing	Track runtime class, self dispatch, super() chain